



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



**Софтверско решење за
двостепенски
ортogonalни проблем ранца са
ротацијом**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Маша Иванов	Број индекса:	SV54/2021
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Софтверско решење за дводимензионални ортогонални проблем ранца са ротацијом

ТЕКСТ ЗАДАТКА:

Дизајнирати и имплементирати софтверско решење за дводимензионални ортогонални проблем ранца са дозвољеном ротацијом правоугаоника за 90 степени. Реализовати генетски алгоритам за избор подскупа правоугаоника који максимизује искоришћену површину и минимизује празан простор на задатој површини. Декодирање хромозома треба реализовати помоћу више хеуристика за постављање правоугаоника и обезбедити механизам за њихово поређење. Потребно је реализовати визуализацију решења као и поређење резултата са другим хеуристикама. При изради користити препоручену праксу из области софтверског инжењерства. Детаљно документовати решење.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА	

Редни број, РБР:		
Идентификациони број, ИБР:		
Тип документације, ТД:		Монографска документација
Тип записа, ТЗ:		Текстуални штампани материјал
Врста рада, ВР:		Дипломски - бечелор рад
Аутор, АУ:		Маша Иванов
Ментор, МН:		Др Игор Дејановић, редовни професор
Наслов рада, НР:		Софтверско решење за дводимензионални ортогонални проблем ранца са ротацијом
Језик публикације, ЈП:		српски/ћирилица
Језик извода, ЈИ:		српски/енглески
Земља публиковања, ЗП:		Република Србија
Уже географско подручје, УГП:		Војводина
Година, ГО:		2026
Издавач, ИЗ:		Ауторски репринт
Место и адреса, МА:		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страница/ црта/табела/слика/графика/прилога)		4/43/9/1/16/0/2
Научна област, НО:		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:		2D ортогонални knapsack проблем, генетски алгоритам, хеуристике
УДК		
Чува се, ЧУ:		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:		
Извод, ИЗ:		Овај рад решава 2D ортогонални <i>knapsack</i> проблем са ротацијом правоугаоника за 90 степени применом генетског алгоритма. Декодирање хромозома реализовано је хеуристикама <i>maximal rectangle</i> , <i>guillotine</i> и <i>skyline</i> , уз могућност поређења резултата. Апликација подржава ручни унос и JSON, као и визуелизацију распореда. Тестови на GCUT инстанцама показују да <i>maximal rectangle</i> најчешће даје највећу искоришћеност, <i>guillotine</i> је најбржи, а <i>skyline</i> представља компромис.
Датум прихватања теме, ДП:		
Датум одбране, ДО:		17.07.2026
Чланови комисије, КО:	Председник:	Др Бранко Милосављевић, редовни професор
	Члан:	Др Гордана Милосављевић, редовни професор
	Члан:	
	Члан, ментор:	Др Игор Дејановић, редовни професор
		Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Maša Ivanov	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	A software solution for the two-dimensional orthogonal knapsack problem with rotation	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	4/43/9/1/16/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	2D orthogonal knapsack problem, genetic algorithm, heuristics	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This thesis solves the 2D orthogonal knapsack problem with 90-degree rectangle rotation using a genetic algorithm. Chromosome decoding is implemented with the maximal rectangle, guillotine, and skyline heuristics, with the ability to compare results. The application supports manual input and JSON, as well as layout visualization. Tests on GCUT instances show that maximal rectangle most often achieves the highest utilization, guillotine is the fastest, and skyline is a compromise.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	17.07.2026	
Defended Board, DB :	President: Branko Milosavljević, Phd., full professor Member: Gordana Milosavljević, Phd., full professor Member: Member, Mentor: Igor Dejanović, Phd., full professor	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Проблем ранца	1
1.2	Значај и примена 2D проблема паковања и сечења	1
1.3	Стање у области	2
1.4	Изазови при решавању проблема	2
1.5	Циљеви рада	2
2	Имплементација решења	5
2.1	Архитектура система	5
2.2	Унос података	7
2.3	Генетски алгоритам	7
2.4	Хеуристике	11
3	Кориснички интерфејс	21
3.1	Почетни екран	21
3.2	Екран за унос преко JSON фајла	22
3.3	Екран за избор хеуристике	23
3.4	Екран за ручни унос	24
3.5	Екран за приказ резултата	25
3.6	Екран за поређење хеуристика	26
4	Закључак	29
	Списак слика	31
	Списак листинга	33
	Списак табела	35
	Списак коришћених скраћеница	37
	Списак коришћених појмова	39
	Биографија	41
	Литература	43

Тема овог рада је проблем постављања правоугаоника на правоугаону површину, односно варијанта проблема ранца, чија је формална дефиниција дата у наредном одељку. За решавање овог проблема развијена је апликација која корисницима омогућава примену различитих алгоритама и хеуристика.

1.1 Проблем ранца

Проблем који се решава је проблем ранца (енг. *knapsack*), који се односи на постављање правоугаоника на правоугаону површину са следећим ограничењима:

- Правоугаоници не смеју да се преклапају нити да излазе из граница површине.
- Дозвољено је ротирање правоугаоника за 90 степени.
- Постављање свих правоугаоника није обавезно.

Циљ је да се поставе правоугаоници тако да остане што мање слободног простора на површини. Овај проблем је важан у различитим областима, као што су индустрија, логистика и рачунарство, јер помаже у оптимизацији простора и ресурса. У овом раду се разматра варијанта проблема у којој се вредност правоугаоника изједначава са његовом површином, па је циљ максимизација искоришћене површине, односно минимизација празног простора.

Проблем ранца је сродан проблемима паковања (енг. *bin packing*) и сечења (енг. *cutting stock*), али се од њих разликује по циљу оптимизације. Код проблема ранца број површина је фиксиран, најчешће једна површина, а бира се подскуп правоугаоника који даје највећу укупну вредност или, у варијанти коришћеној у овом раду, највећу искоришћену површину. Код проблема паковања циљ је да се сви задати предмети распореде у што мањи број контејнера. Код проблема сечења предмети су често груписани кроз захтеве, односно вредности захтева (енг. *demand*), а циљ је да се задовоље тражене количине уз што боље коришћење расположивог материјала. Због тога се проблем ранца може посматрати као проблем избора најбољег подскупа предмета за једну ограничену површину, док проблеми паковања и сечења више наглашавају распоређивање задатих предмета или испуњавање захтева [\[1\]](#).

1.2 Значај и примена 2D проблема паковања и сечења

Проблеми 2D паковања и сечења имају практичну примену у индустрији обраде материјала где је неопходно постићи ефикасно коришћење материјала и минимизирати отпад. На пример, у текстилној индустрији, оптимално распоређивање узорака на

тканину може значајно смањити количину отпада. У производњи метала, дрвета или пластике, правилно сечење материјала може довести до значајних уштеда. Такође, ови проблеми се јављају у логистици и складиштењу, где је важно максимално искористити доступан простор за складиштење предмета различитих величина.

1.3 Стање у области

Постоје бројни алати за решавање проблема паковања и сечења, како отвореног кода (енг. *open-source*) тако и комерцијални, али ниједан од њих не комбинује генетски алгоритам, вишеструке декодирајуће хеуристике и визуелно поређење резултата хеуристика у оквиру једног система.

Од алата отвореног кода (енг. *open-source*) најпознатији су библиотека *rectpack*¹, *SVGNest*² и његов наследник *Deepnest*³. У *Rust* екосистему, најнапреднији алат је *sparrow*⁴, а од комерцијалних алата најпознатији је *SigmaNest*⁵. Сви наведени алати нуде само један приступ решавању проблема, више хеуристика без генетског алгоритма или генетски алгоритам са само једном стратегијом постављања. Ниједан од њих не омогућава да се унутар исте апликације исти проблем реши различитим хеуристикама и да се резултати упореде.

1.4 Изазови при решавању проблема

Решавање проблема ранца и сродних 2D проблема паковања и сечења доноси низ изазова. Један од главних изазова је велики број могућих комбинација избора и распореда правоугаоника, што чини проблем комбинаторно сложеним, а ограничења наведена у претходном одељку (1.1) додатно компликују процес решавања. Осим тога, потребно је узети у обзир различите хеуристике и стратегије које могу утицати на квалитет решења, као и ефикасност алгоритама који се користе за проналажење оптималних или приближних решења. Проблем је и време извршавања, са повећањем броја правоугаоника и величине површине расте и време потребно за проналажење решења, што захтева оптимизацију алгоритама и коришћење ефикасних метода за смањење сложености проблема.

1.5 Циљеви рада

Рад је намењен свима који су заинтересовани за решавање проблема ранца како би се упознали са генетским алгоритмом и различитим хеуристикама. Рад је погодан и за оне који немају предзнање из ове области, зато што им апликација може послужити као алат за решавање проблема ранца.

¹<https://github.com/secnot/rectpack>

²<https://github.com/Jack000/SVGNest>

³<https://github.com/Jack000/Deepnest>

⁴<https://github.com/JeroenGar/sparrow>

⁵<https://www.sigmanest.com/en/sigmanest>

Циљеви рада су:

- имплементација генетског алгоритма за решавање проблема ранца,
- имплементација више декодирајућих хеуристика за постављање правоугаоника на површину,
- визуелизација решења и поређење резултата различитих хеуристика,
- пружање могућности корисницима да унесу своје податке и тестирају различите стратегије решавања проблема ранца.

Глава 2

Имплементација решења

За имплементацију решења коришћен је програмски језик *Rust*, због високих перформанси и нивоа контроле над подацима што омогућава имплементацију сложених алгоритама.

2.1 Архитектура система

Апликација је реализована као један *Rust crate*, унутар кога је функционалност подељена на више модула. Сваки модул је смештен у посебан изворни фајл и има јасно дефинисану улогу у систему. Имплементирани модули су: *genetic*, *max_rects*, *skyline*, *guillotine*, *ui*, *menu_state* и *util*.

Улазна тачка апликације је *main.rs*. Унутар њега се декларишу остали модули и покреће се *AppState*. *Main* такође садржи структуру *Problem* и енумерацију *Heuristic*. Структура *Problem* садржи величине површине и правоугаоника који се постављају на површину.

Модули се могу поделити у три групе према нивоу архитектуре: алгоритамски, презентациони и помоћни слој.

Алгоритамском слоју припадају модули *genetic*, *max_rects*, *skyline* и *guillotine*. Сваки од њих представља имплементацију алгорита који одговара називу модула. Енумерација *Heuristic* омогућава да се свака хеуристика користи кроз исти интерфејс *decode_chromosome*. Тиме се омогућава да генетски алгоритам не мора да зна детаље имплементација хеуристика, већ само да позове одговарајућу хеуристику на основу вредности енумерације.

```
pub enum Heuristic {
    MaxRects,
    Skyline,
    Guillotine,
}

impl Heuristic {
    pub fn decode_chromosome(&self, chromosome: &[u8], problem: &Problem)
        -> (Vec<Rect>, f32) {
        match self {
            Heuristic::MaxRects =>
max_rects::decode_chromosome(chromosome, problem),
            Heuristic::Skyline   => skyline::decode_chromosome(chromosome,
problem),
            Heuristic::Guillotine =>
guillotine::decode_chromosome(chromosome, problem),
        }
    }
}
```

Листинг 1: Апстракција хеуристике преко енумерације.

Презентационом слоју припадају модули *ui* и *menu_state*. Унутар модула *ui* имплементирани су основне компоненте графичког интерфејса, као што су дугмад и текстуални уноси. Модул *menu_state* представља стање апликације, навигацију кроз меније, као и читавање и покретање решења. Стање апликације је представљено кроз енумерацију *MenuState* која садржи сва могућа стања апликације. Једно стање апликације одговара једном екрану за приказ и једној фази рада програма. Оваква организација омогућава да се логика приказа одвоји од логике решавања проблема, чиме се поједностављује одржавање кода и смањује могућност појаве грешака при навигацији. Модул *menu_state* такође садржи и структуре *AppState* и *HeuristicResult*. Структура *AppState* чува податке неопходне за рад програма као што су изабрана хеуристика, учитани проблеми, резултати решавања и привремени подаци за ручни унос. Структура *HeuristicResult* чува резултате решавања проблема као што су одабрана хеуристика, време извршавања, проценат искоришћености површине и број постављених правоугаоника.

Помоћном слоју припада модул *util*. Унутар овог модула се налазе заједничка структура *Rect* и функције које се користе у више модула. Структура *Rect* представља правоугаоник и садржи његове димензије и координате горњег левог угла. Структура у себи садржи и методе за основне операције као што су израчунавање површине, провера садржаности и преклапања правоугаоника, чиме се омогућава лакши рад са правоугаоникима кроз све модуле апликације.

2.2 Унос података

Корисник уноси димензије површине као и димензије правоугаоника који се на њу постављају. Унос је могућ преко графичког интерфејса или преко JSON фајла. Ширине као и висине правоугаоника морају бити позитивни цели бројеви. Уколико је изабран унос преко JSON фајла, корисник може да изабере један од 13 унапред учитаних GCUT проблема или да прочита податке из свог фајла.

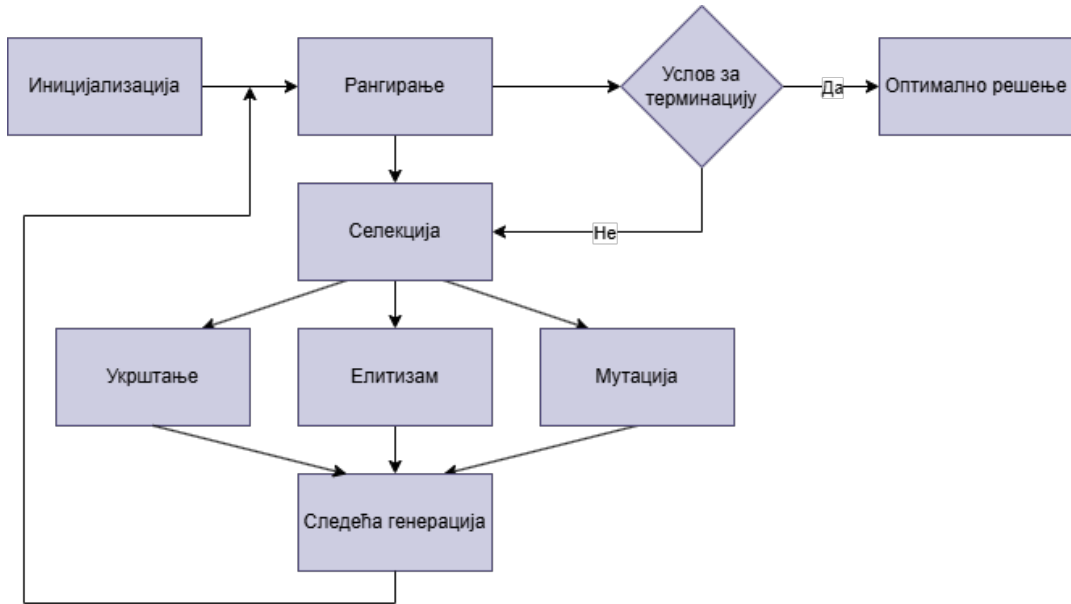
```
{
  "Objects": [
    { "Length": 100, "Height": 80 }
  ],
  "Items": [
    { "Length": 30, "Height": 20, "Demand": 2 },
    { "Length": 10, "Height": 40, "Demand": 1 },
    { "Length": 25, "Height": 25 }
  ]
}
```

Листинг 2: Пример JSON фајла.

Objects представља површину и њене димензије, док *Items* представљају правоугаонике који се постављају на површину. Како не би дошло до понављања уноса, сваки правоугаоник има опционо поље *Demand* које представља количину правоугаоника истих димензија. Уколико се поље *Demand* не налази, подразумева се да се тај правоугаоник појављује једном.

2.3 Генетски алгоритам

Тачне методе могу претраживати велики простор могућих распореда и зато често имају високу временску сложеност, због тога се у овом раду користи генетски алгоритам као хеуристички приступ. Генетски алгоритам постепено тежи ка оптималном решењу, док се неповољна решења током еволуције одбацују [2]. На слици 1 приказана је архитектура генетског алгоритма.



Слика 1: Архитектура генетског алгоритма.

Генетски алгоритам се састоји од седам фаза: иницијација, рангирање, селекција, репродукција, укрштање, мутација, елитизам и терминација.

2.3.1 Параметри

Параметри генетског алгоритма утичу на квалитет решења и време извршавања.

Величина популације одређује колико јединки ће се налазити у свакој генерацији. Већа популација може довести до бољих решења, али и до дужег времена извршавања. У овом раду, величина популације је постављена на 100.

Вероватноћа мутације одржава генетску разноликост и представљена је процентом који одређује колико ће се битова у хромозому променити током мутације. Већа вероватноћа мутације може помоћи у избегавању локалних оптимума, али превисока може довести до хаотичног понашања алгоритма. У овом раду, вероватноћа мутације је постављена на 10%.

Елитизам обезбеђује да се најбоље јединке из генерације директно преносе у наредну генерацију без икаквих промена, али превелик елитизам смањује разноликост у популацији. У овом раду, проценат јединки које се чувају као елита је постављен на 10%.

Максималан број генерација одређује када ће се алгоритам зауставити. Већи број генерација може довести до бољих решења, али и до дужег времена извршавања. У овом раду, максималан број генерација је постављен на 200.

2.3.2 Иницијализација

У фази иницијације, генерише се почетна популација која се састоји од конфигурисаног броја јединки. Јединка представља могуће решење проблема и састоји се од бинарног низа дужине једнаке броју правоугаоника. Свака позиција у низу представља један правоугаоник, а вредност позиције представља да ли се тај правоугаоник налази на површини или не. Вредност 1 означава да се правоугаоник налази на површини, док вредност 0 означава да се не налази. Јединке почињу као низ случајних јединица и нула, с тим да је однос јединица и нула унапред одређен.

```
pub fn generate_chromosome(len: usize, rng: &mut StdRng) -> Vec<u8> {
    let mut chromosome = vec![1u8; len];
    let zeros = len / 10;

    (0..zeros).for_each(|_| {
        chromosome[rng.random_range(0..len)] = 0;
    });
    chromosome
}
```

Листинг 3: Иницијализација хромозома.

2.3.3 Рангирање

Рангирање се врши на основу вредности *fitness* функције која се израчунава за сваки декодирани хромозом. Декодирање хромозома се врши изабраном хеуристиком. *Fitness* функција се дефинише као однос искоришћеног и неискоришћеног дела површине на којој су постављени правоугаоници. Јединке са већим *fitness*-ом имају веће шансе да буду изабране за селекцију и формирање наредне популације.

Рангирање чини уско грло (енг. *bottleneck*) генетског алгоритма, јер се *fitness* функција мора израчунати за сваку јединку у популацији, што може бити временски захтевно, посебно ако је број правоугаоника велики. Због тога се ова фаза убрзава тако што се *fitness* паралелно израчунава за више јединки истовремено, користећи више процесорских језгара. Библиотека коришћена за паралелизацију је *rayon*, а паралелизација се реализује итерацијом кроз све хромозоме популације.

2.3.4 Селекција

Селекција се врши помоћу рулетске селекције (енг. *roulette-wheel selection*). Детерминистичке методе селекције се генерално не користе јер често не могу да премосте локални оптимум. Рулетска селекција насумично бира родитеље узимајући у обзир *fitness* вредности, тако да јединке са већим *fitness*-ом имају веће шансе да буду изабране [3]. Изабрани родитељи се затим користе за генерисање потомака у фази укрштања.

2.3.5 Репродукција

Репродукција представља процес додавања потомака у нову популацију. Потомци се добијају укрштањем, мутацијом и елитизмом. Поступак репродукције се понавља до терминације алгоритма.

```
for _iteration in 0..max_iterations {
  let ranked = rank_chromosomes(&population, problem, heuristic);
  if ranked[0].1 > best_fitness {
    best_fitness = ranked[0].1;
    best_chromosome = ranked[0].0.clone();
  }
  let parents: Vec<Vec<u8>> = ranked.iter().map(|(chr, _)|
chr.clone()).collect();
  let pairs = roulette_selection(&ranked, rng);
  let children = two_point_crossover(&pairs, rng);
  let mutated_children = mutation(&children, mutation_rate, rng);
  population = elitism(&parents, &mutated_children, elitism_rate,
population_size);
}
```

Листинг 4: Петља генетског алгоритма.

2.3.6 Укрштање

Укрштање се врши помоћу укрштања у две тачке (енг. two-point crossover). Насумично се бирају две тачке у низу јединки родитеља, а потом се размењују делови између тих тачака између два родитеља. Овај процес генерише два потомка који садрже комбинацију особина оба родитеља. Од првог родитеља се узима део од почетка до прве тачке, затим се узима део између две тачке од другог родитеља, а на крају се узима део од другог родитеља од друге тачке до краја. За другог потомка се ради супротно. На слици 2 приказано је укрштање у две тачке између два родитеља и добијање два потомка.



Слика 2: Укрштање у две тачке.

2.3.7 Мутација

Мутација се врши како би се одржала генетска разноликост у популацији и спречила стагнација у локалном оптимуму. Врста мутације која се врши је окретање бита (енг. bit flip). За сваку позицију у низу јединке постоји одређена вероватноћа да ће се променити вредност позиције. Уколико је вредност позиције 1, она се мења у 0, а ако је вредност позиције 0, она се мења у 1.

2.3.8 Елитизам

Како не би дошло до губитка најбољих решења, користи се елитизам. Елита се састоји од предефинисаног броја јединки са највећим *fitness*-ом које се директно преносе у нову популацију без икаквих промена. Остатак нове популације попуњава потомцима који су настали мутацијом и укрштањем.

2.3.9 Терминација

Терминација се дешава када се достигне максималан број генерација, који је постављен на 200. Уколико се у том тренутку не пронађе оптимално решење, алгоритам се зауставља и враћа најбоље пронађено решење.

2.4 Хеуристике

Генетски алгоритам ради са једноставним хромозомима, у себи носи само правоугаонике и то да ли су постављени на површини или не. Међутим, само та информација није довољна како би се добило оптимално решење. Због тога се користе хеуристике које изабране правоугаонике постављају на површину. Хеуристике се користе у фази

декодираниа јединки, када се од бинарног низа добија конкретно решење проблема паковања.

Хеуристике које су имплементирание су: *maximal rectangle*, *guillotine* и *skyline*. Ове хеуристике су стандардне за решавање овог типа проблема. Свака од ових хеуристика има различит приступ постављању правоугаоника на површину, а избор хеуристике може утицати на квалитет пронађеног решења. Како обичан корисник не би морао да бира хеуристику, имплементиран је механизам који аутоматски бира хеуристику и проблем решава оном која даје најбоље решење.

```
let heuristics = [Heuristic::MaxRects, Heuristic::Skyline,
Heuristic::Guillotine];
let mut best_result: Option<HeuristicResult> = None;

for heuristic in heuristics {
    let (best_chromosome, fitness) = genetic_algorithm(
        prob, population_size, mutation_rate, elitism_rate, max_iterations,
        rng, heuristic,
    );
    if best_result.as_ref().map_or(true, |b| fitness > b.fitness) {
        best_result = Some(/* ... резултат за ову хеуристику ... */);
    }
}
```

Листинг 5: Аутоматски избор хеуристике.

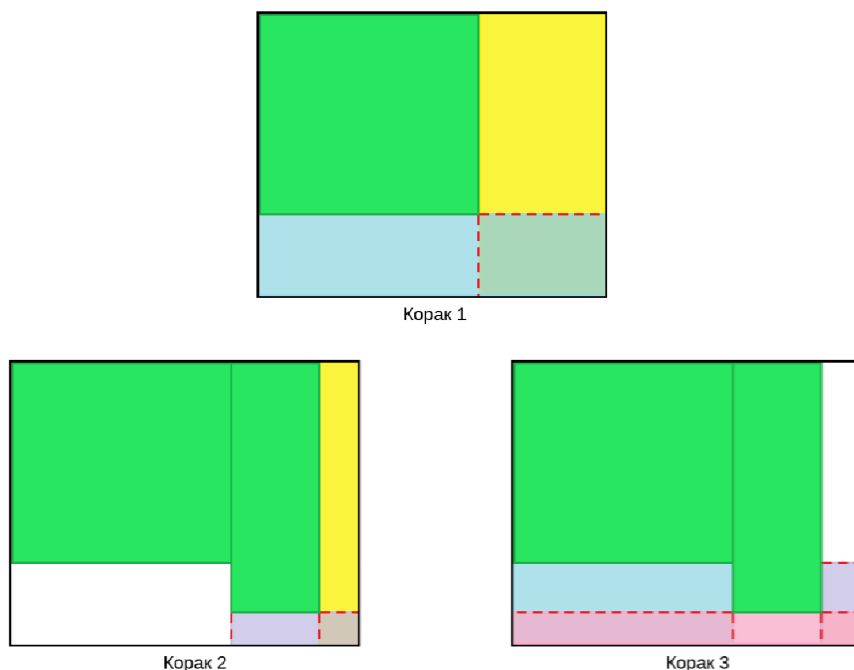
2.4.1 *Maximal rectangle*

Површина на коју се постављају правоугаоници се на почетку представља као један велики правоугаоник. Пролази се кроз хромозоме за сваки ген чија је вредност 1 покушава се постављање одговарајућег правоугаоника.

Најбоље место се одређује тестирањем свих тренутно слободна места на којима се правоугаоник може поставити, у нормалној и ротираној оријентацији (ротација од 90 степени). Правоугаоник се може поставити на слободно место ако су његове димензије мање или једнаке димензијама слободног места. Кандидат се бира тако да има најмањи отпад на краћој страни, затим најмањи отпад на дужој страни, а уколико је и то исто, бира се решење са најмањим преосталим отпадом по површини.

Након избора кандидата проверава се да правоугаоник остаје у границама површине и да се не преклапа са већ постављеним правоугаоникима; у супротном се прескаче. Правоугаоник се поставља у горњи леви угао слободне површине. Када се правоугаоник успешно постави, пролази се кроз листу слободних површина и свака површина која се преклапа са новопостављеним правоугаоником се уклања и замењује са највише 4 нове слободне површине (лево, десно, изнад и испод постављеног правоугаоника).

На слици 3 приказано је дељење површине након постављања правоугаоника. Зеленом бојом су приказани правоугаоници који су постављени на површину док су другим бојама приказане слободне површине које су настале након постављања правоугаоника. Црвеним испрекиданим линијама показане су ивице правоугаоника. На првом кораку на слици је постављен један правоугаоник са 2 слободне површине, жутом и плавом бојом, које имају преклапање у доњем десном углу. На другом кораку, нови правоугаоник је постављен на жуту слободну површину, што је довело до дељења жуте површине на две нове слободне површине, жуту и љубичасту које се преклапају. На трећем кораку је приказано како се дели плава слободна површина након постављања другог правоугаоника. Плава површина се дели на три нове слободне површине, плаву, љубичасту и црвену које се преклапају. Све слободне површине које се добију након постављања другог правоугаоника се добију као скуп новонасталих слободних површина након дељења жуте и плаве површине, с тим да ће љубичаста површина бити уклоњена јер је у целисти садржана у црвеној површини.



Слика 3: Пример дељења површине.

Због начина креирања слободних површина, оне се могу преклапати. Након постављања правоугаоника потребно је проћи кроз листу слободних површина и уклоњити

оне које су у целости садржане у некој другој површини, како би се смањио број кандидата и убрзао рад алгоритма.

Fitness функција све три хеурисџике се израчунава на исти начин, као однос површине коју заузимају правоугаоници који се налазе на површини и укупне површине.

- Предности:
 - Често даје боље резултате.
 - Детаљнија евалуација кандидата.
 - Добро ради са правоугаоникима који имају велике разлике у димензијама.
- Недостаци:
 - Време извршавања је дуже због ажурирања листе слободних површина након постављања правоугаоника.
 - Број слободних површина може постати велики.
 - Комплекснија обрада слободних површина које се преклапају.

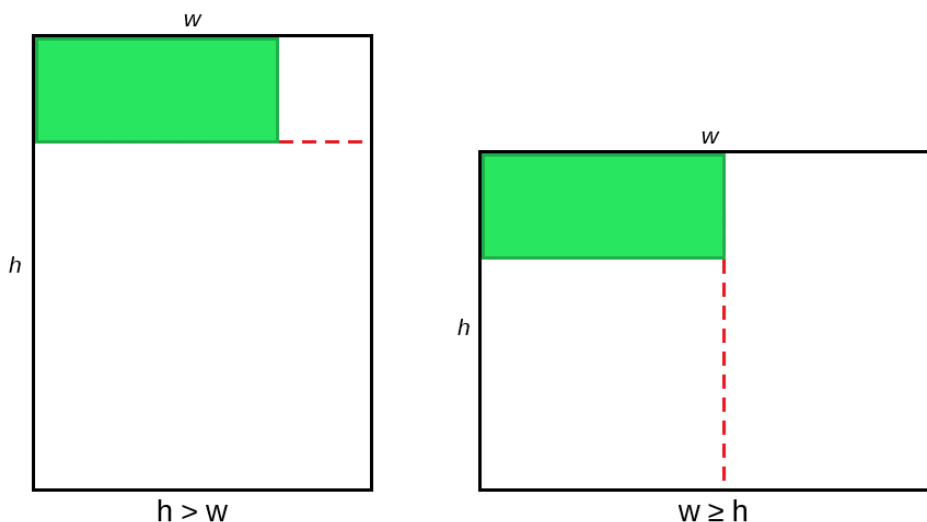
2.4.2 Guillotine

Почетна површина као и проналажење најбољег места ради се на исти начин као код *maximal rectangle* хеурисџике. Разлика је у третирању преостале слободне површине након постављања правоугаоника, и у томе што се не гледа решење са најмањим отпадом, већ се увек бира прво пронађено решење.

Први правоугаоник се поставља увек у горњи леви угао почетне слободне површине. За наредне правоугаонике се проналази најбоље место, а правоугаоник се поставља на горњи леви угао изабране.

Уколико се правоугаоник не може поставити ни на једно слободно место (преклапа се са већ постављеним правоугаоником или је ван граница површине), он се прескаче.

Након постављања правоугаоника, преостала слободна површина унутар површине на којој је постављен правоугаоник се дели на два дела помоћу једне гиљотинске секуће линије. Гиљотинска линија може бити хоризонтална или вертикална, а избор линије се врши на основу тога која димензија слободне површине (на којој је управо постављен правоугаоник) је већа, ширина или висина. Уколико је ширина већа или једнака висини, ради се вертикално сечење површине и тиме се добијају две нове слободне површине, десни и доњи правоугаоник. У супротном, ради се хоризонтално сечење. На слици 4 приказано је дељење површине гиљотинским сечењем након постављања правоугаоника.



Слика 4: Пример дељења површине гилотинским сечењем.

- Предности:
 - Време извршавања је краће због једноставнијег ажурирања листе слободних површина након постављања правоугаоника.
 - Једноставнија обрада слободних површина.
 - Перформансе су предвидиве због линеарне комплексности.
- Недостаци:
 - Често не изабере најбоље место, него прво место које пронађе, што може довести до лошијих решења.
 - Лоше се прилагођава правоугаонцима који имају велике разлике у димензијама, јер увек сече по већој димензији, што може довести до неефикасног коришћења простора.
 - Може довести до фрагментације слободног простора, што отежава постављање наредних правоугаоника.

2.4.3 Skyline

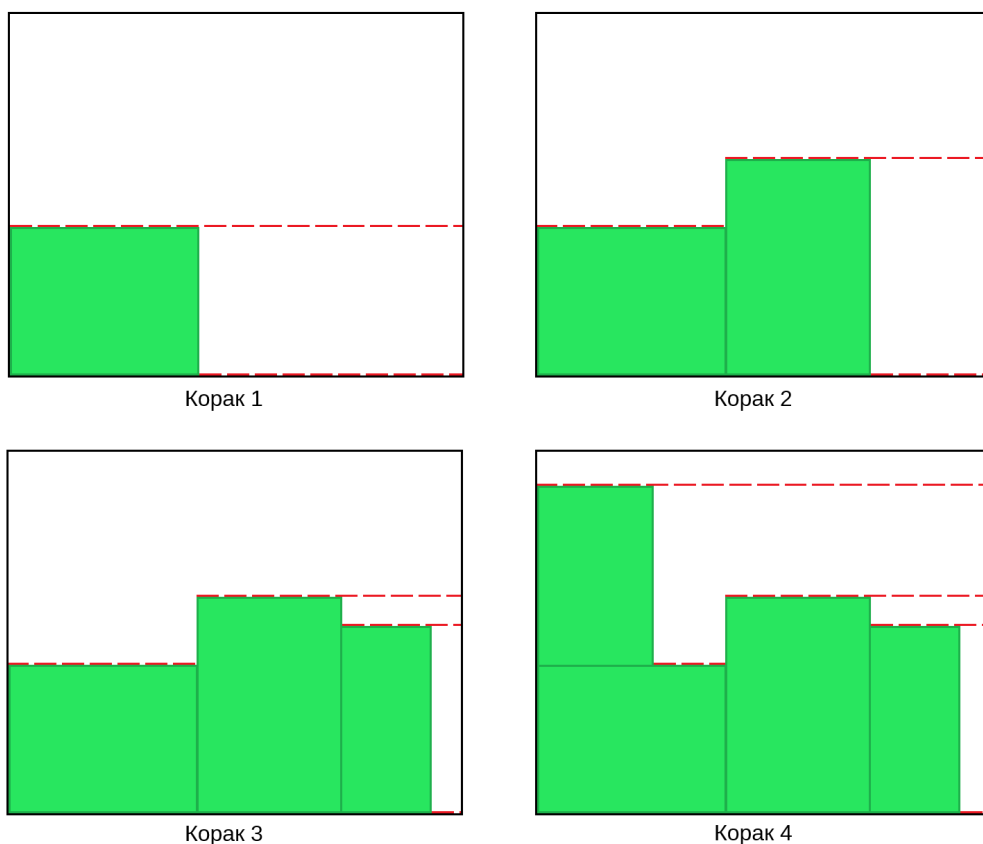
Skyline представља листу хоризонталних сегмената који описују тренутни хоризонт слободне површине. Сегмент се дефинише са три вредности: почетна координата, крајња координата (која се рачуна тако што се саберу почетна координата и ширина сегмента) и висина. На почетку постоји један сегмент који представља целу површину са висином 0. За сваки правоугаоник чија је вредност гена 1, проналази се најбоље место за постављање правоугаоника.

Најбоље место се одређује тако што се за сваки сегмент, и за обе ротације правоугаоника, израчунава висина на коју би правоугаоник био постављен. Та висина је максимум висина свих сегмената који преклапају хоризонтални распон правоугаоника, а као допунски критеријум користи се укупан отпад испод те висине (разлика између максималне висине и појединачних сегмената у том распону). Кандидат се бира тако да има најмању висину постављања, а при истој висини најмањи отпад.

Уколико се правоугаоник не може поставити ни на једно слободно место (преклапа се са већ постављеним правоугаоником или је ван граница површине), он се прескаче.

Након успешног постављања ажурира се листа сегмената. Сегменти који су у потпуности лево или десно од постављеног правоугаоника остају непромењени. Сегменти који се хоризонтално преклапају (опсежи сегмента и правоугаоника на x оси се секу) са постављеним правоугаоником се деле на леви и/или десни остатак. Затим се додаје нови сегмент на висини врха правоугаоника, преко његове ширине. На крају се сегменти сортирају по x и спајају се суседни сегменти исте висине како би листа остала компактна.

На слици 5 приказани су сегменти након постављања правоугаоника. Зеленом бојом приказани су постављени правоугаоници, црвеним испрекиданим линијама приказани су сегменти. На првом кораку постављен је један правоугаоник и тиме добијамо два сегмента, један са висином 0 и други са висином једнаком висини постављеног правоугаоника. На другом кораку постављен је други правоугаоник на најнижи сегмент и он се преклапа са оба сегмента, па се добијају три сегмента, од којих је један са висином 0, други са висином једнаком висини другог постављеног правоугаоника, а трећи са висином једнаком висини првог постављеног правоугаоника. На трећем кораку постављен је трећи правоугаоник на најнижем сегменту (сегменту висине 0), па се тај сегмент дели на два сегмента, један са висином 0 и други са висином једнаком висини трећег постављеног правоугаоника, и остају два сегмента која су већ постојала, висина претходно постављених правоугаоника. На четвртом кораку постављен је четврти правоугаоник на најнижем сегменту где је могао бити постављен (на сегменту висине првог постављеног правоугаоника), па се тај сегмент дели на два сегмента, један са висином једнаком висини последњег постављеног правоугаоника и други са висином једнаком висини првог постављеног правоугаоника, и остају три сегмента која су већ постојала, висина претхно постављених правоугаоника и сегмент висине 0.



Слика 5: Пример сегмената након постављања правоугаоника.

- Предности:
 - Брзо ради са правоугаоникима великих димензија.
 - Погодан за проблеме где се правоугаоници доста разликују по висини.
 - Средња комплексност, балансован квалитет са брзином извршавања.
- Недостаци:
 - Може бити мање ефикасан са малим правоугаоникима или квадратима.
 - Број сегмената може постати велик.
 - Фрагментација слободног простора је мање предвидива.

2.4.4 Поређење хеуристика

GCUT је скуп стандардних *Beasley* инстанци за неограничени дводимензионални проблем гиљотинског сечења, које се често користе као *benchmark* у области дводимензионалног сечења и паковања [4]. Сваки проблем дефинише једну површину и листу правоугаоника са целобројним димензијама, па су погодни за поређење

хеуристика. Хеуристике су тестиране на 13 GCUT проблема различитих величина, од најмањег *gcut1* до највећег *gcut13*. *Gcut1* проблем има површину од 250x250 и 10 правоугаоника средњих димензија (ширина и висина између 70 и 180). *Gcut5* је проблем средње величине са површином од 500x500 и 10 већих правоугаоника. *Gcut13* је највећи проблем са површином од 3000x3000 и 32 правоугаоника, укључујући неке веома високе крупне елементе, што повећава сложеност.

Резултати решавања *gcut1-gcut13* проблема помоћу све три хеуристике су приказани у табели 1. Приказани су проценти искоришћености површине као и време извршавања за сваки проблем и хеуристику. Проценти искоришћености су израчунати као однос површине коју заузимају правоугаоници који се налазе на површини и укупне површине, а време извршавања је мерено у милисекундама. Што је већи проценат искоришћености, то је боље решење и има мање отпада.

Табела 1: Поређење хеуристика на 13 *gcut* проблема.

Проблем	<i>Maximal rectangle</i>		<i>Guillotine</i>		<i>Skyline</i>	
	Искоришћеност	Време извршавања	Искоришћеност	Време извршавања	Искоришћеност	Време извршавања
<i>gcut1</i>	93%	16ms	93%	15ms	93%	16ms
<i>gcut2</i>	96.7%	17ms	94.9%	15ms	96.7%	17ms
<i>gcut3</i>	96.2%	18ms	95.8%	16ms	96.2%	18ms
<i>gcut4</i>	97.8%	20ms	96.1%	18ms	97.7%	21ms
<i>gcut5</i>	86.3%	16ms	93.6%	15ms	89.7%	16ms
<i>gcut6</i>	94.8%	18ms	95.2%	15ms	94.8%	16ms
<i>gcut7</i>	97%	18ms	96.8%	17ms	93.7%	17ms
<i>gcut8</i>	97.9%	20ms	96.5%	18ms	97%	20ms
<i>gcut9</i>	95.3%	17ms	91.9%	15ms	95.3%	16ms
<i>gcut10</i>	93.8%	16ms	91.2%	15ms	90.8%	16ms
<i>gcut11</i>	95.7%	18ms	94.8%	16ms	95.7%	18ms
<i>gcut12</i>	96.8%	19ms	95.8%	17ms	96.8%	19ms
<i>gcut13</i>	89.8%	37ms	90.8%	29ms	85.7%	18ms

У спроведеним тестовима *maximal rectangle* у већини *gcut* проблема даје највећу искоришћеност, али уз нешто дуже време извршавања, што је очекивано због сложенијег ажурирања слободних површина. *Guillotine* је у просеку најбржи, али чешће губи

на квалитету решења јер често узима прво погодно слободно место за постављање правоугаоника; изузетак су проблеми као *gcut5* и *gcut6*, где даје најбоље или веома конкурентне резултате. *Skyline* углавном постиже средње резултате, на неким инстанцама је близак најбољем (нпр. *gcut2*, *gcut3*, *gcut9*, *gcut11*), док на другима заостаје.

Из тога следи да избор хеуристике зависи од приоритета: ако је циљ највећа искористљеност, *maximal rectangle* је најчешће најбољи избор; ако је време извршавања критично, *guillotine* даје најбрже резултате; ако је потребан компромис, *skyline* је стабилна средња опција. Због тога је имплементиран механизам који аутоматски бира хеуристику са најбољим резултатом за дати проблем.

Након што се проблем реши, кориснику се приказује визуелизација решења и даје му се опција да упореди резултате свих хеуристика.

Глава 3

Кориснички интерфејс

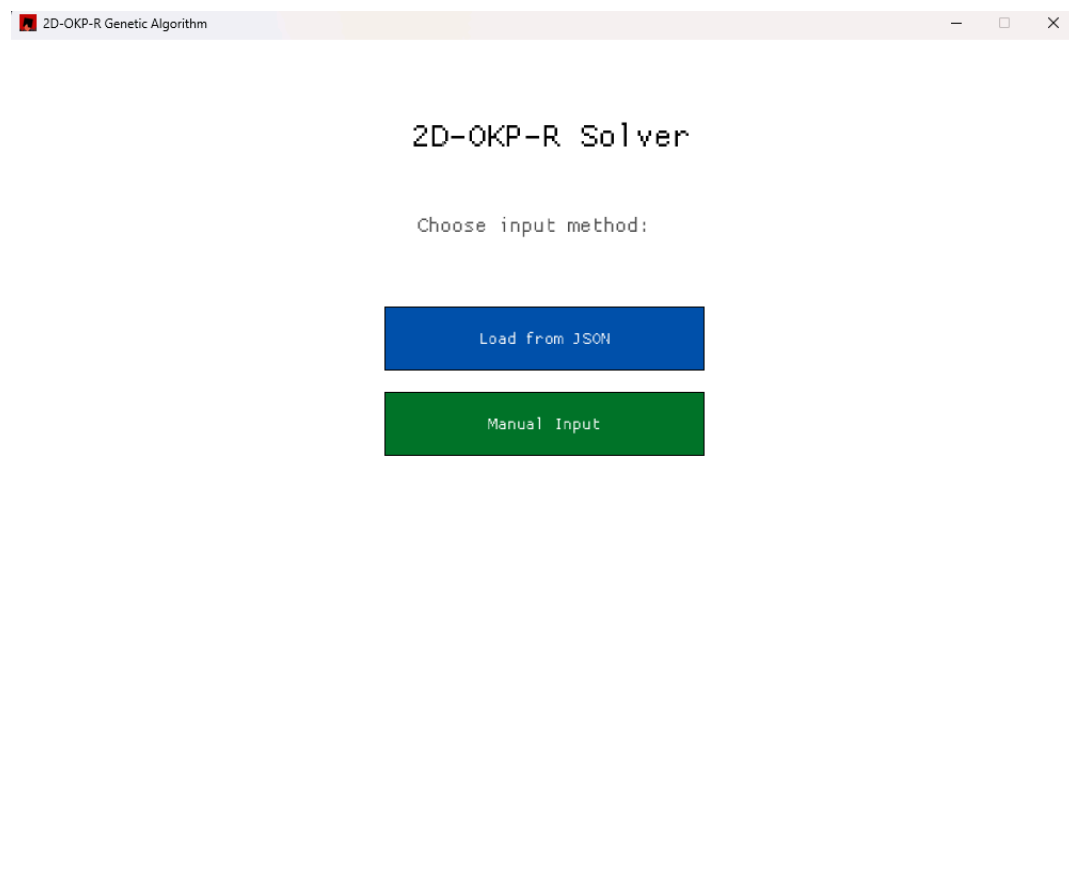
Кориснички интерфејс је креиран помоћу *macroquad* библиотеке. Интерфејс је осмишљен тако да буде једноставан и интуитиван, са јасним опцијама за унос података, покретање алгоритма и приказ резултата.

Визуелни елементи су минималистички, са фокусом на функционалност. Интерфејс користи основне геометријске облике и текст за приказ информација, а боје су коришћене за истицање важних елемената као што су дугмад и резултати. Најчешће коришћени елементи су дугмад и поља за унос текста, који су јасно означени и лако доступни. Структура интерфејса је подељена на неколико екрана: почетни екран за избор начина уноса, екран за ручни унос, екран за избор JSON фајла и екран за приказ резултата. На сваком екрану су јасно означене опције и дугмад за навигацију.

Смењивање екрана врши се помоћу једноставне машине стања (енг. *state machine*). Екран који је активан описује се енумерацијом *MenuState*, која има шест вредности, по једну за сваки екран. Главна функција за исцртавање у сваком кадру проверава тренутно стање и позива одговарајућу функцију за исцртавање тог екрана. Навигација се реализује тако што се кликом на дугме мења вредност стања, што доводи до приказивања другог екрана. На овај начин логика је јасно подељена по екранима, а целокупан ток апликације је прегледан и лак за проширење. На сваком екрану се налази “Назад” (енг. *Back*) дугме које враћа корисника на почетни екран, омогућавајући му да лако промени начин уноса или изабере други проблем. Последња два екрана (екран за визуализацију решења и екран за поређење хеуристика) имају и дугме за враћање на почетни екран.

3.1 Почетни екран

Почетни екран је први екран који се приказује када корисник покрене апликацију. На овом екрану корисник има две опције за унос података: ручни унос и унос преко JSON фајла. Одабиром једне од опција, корисник се преусмерава на одговарајући екран за унос података. На слици 6 приказан је почетни екран са обе опције за унос података.



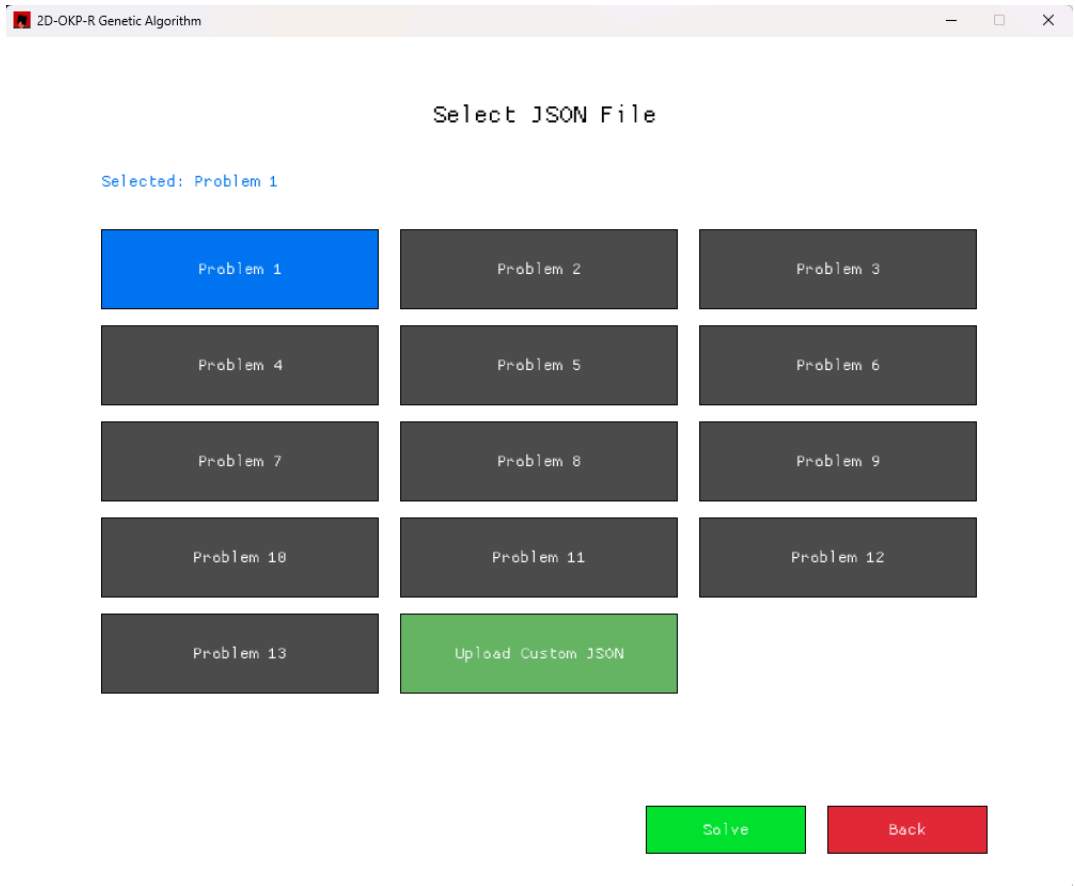
Слика 6: Почетни екран.

3.2 Екран за унос преко JSON фајла

Кориснику је дата опција да изабере један од 13 унапред учитаних GCUT проблема или да уноси податке из свог фајла. Унапред је изабрана опција првог GCUT проблема, кликом на одговарајуће дугме бира други проблем.

Такође постоји дугме за избор JSON фајла са свог рачунара. Када корисник кликне на то дугме, отвара се дијалог за избор фајла.

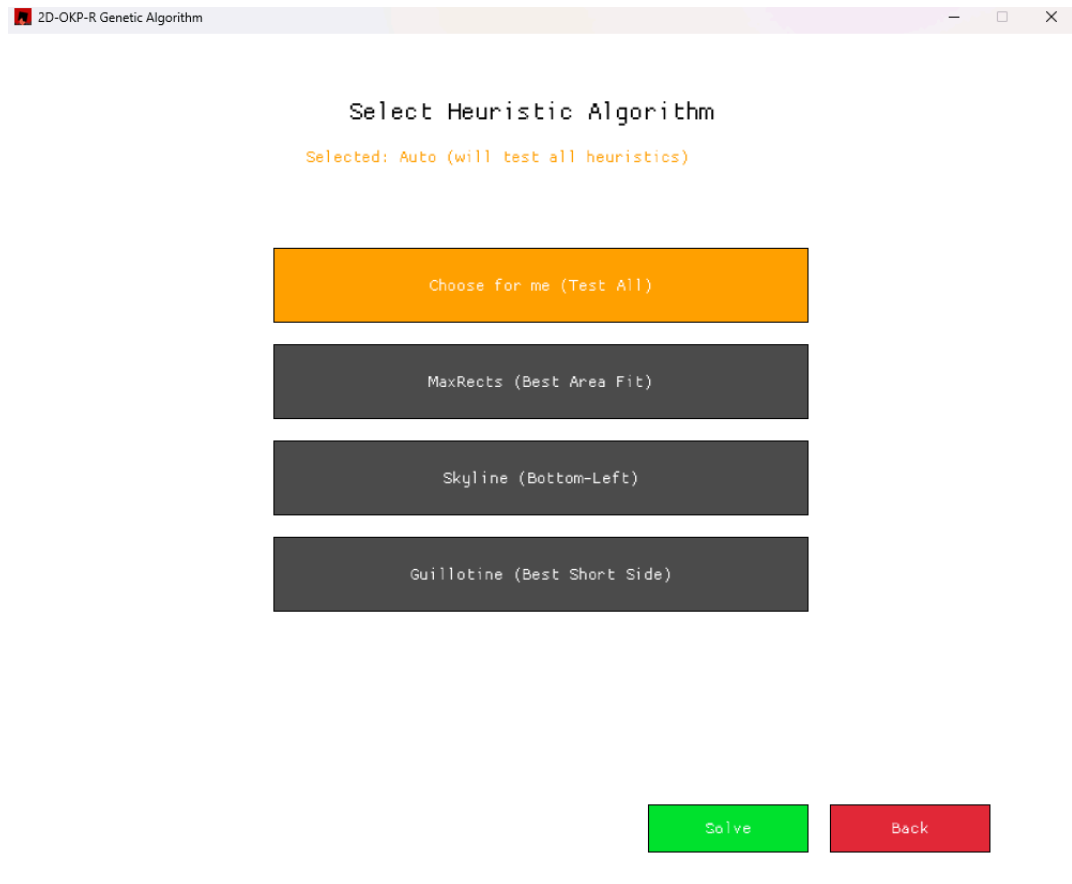
Након одабраног проблема, корисник се кликом на дугме “Реши” (енг. *solve*) преусмерава на екран за избор хеуристике. На слици 7 приказан је екран за унос преко JSON фајла.



Слика 7: Екран за унос преко JSON фајла.

3.3 Екран за избор хеуристике

Екран за избор хеуристике приказује четири дугмета, по једно за сваку хеуристику и једно за аутоматски избор. Испод наслова екрана, укратко је описана тренутно изабрана хеуристика. Корисник се кликом на дугме “Решити” (енг. *solve*) преусмерава на екран за приказ резултата. На слици 8 приказан је екран за избор хеуристике.



Слика 8: Екран за избор хеуристике.

3.4 Екран за ручни унос

На екрану за ручни унос, корисник има поља за унос димензија површине. Пре уноса појединачних правоуганика, потребно је унети количину правоугаоника за постављање. Унутар секције “Димензије правоугаоника” (енг. *rectangle dimensions*) појавиће се поља за унос ширине и висине сваког правоугаоника. Због ограничене величине секције, омогућено је скроловање унутар ње путем миша или стрелица (горе/доле) на тастатури. Такође, кретање између поља за унос је омогућено помоћу табулатора (енг. *tab*) на тастатури. Унос се може обавити и помоћу миша кликом на поље за унос, што ће га активирати и омогућити унос текста.

На истом екрану, корисник бира хеуристику која ће се користити за постављање правоугаоника на површину. Опција “Изабери за мене” (енг. *choose for me*) подразумева да ће алгоритам аутоматски изабрати хеуристику која даје најбоље решење за дати проблем. Корисник такође може ручно одабрати једну од три хеуристике: *maximal rectangle*, *guillotine* или *skyline*.

Уколико корисник жели да крене испочетка уноса, може да кликне на дугме “Обриши” (енг. *clear*) и поново унесе податке.

Након уноса свих потребних података, корисник кликом на дугме “Реши” (енг. *solve*) преусмерава се на екран за приказ резултата. На слици 9 приказан је екран за ручни унос са примером када је потребно унети 10 правоугаоника.

Слика 9: Екран за ручни унос.

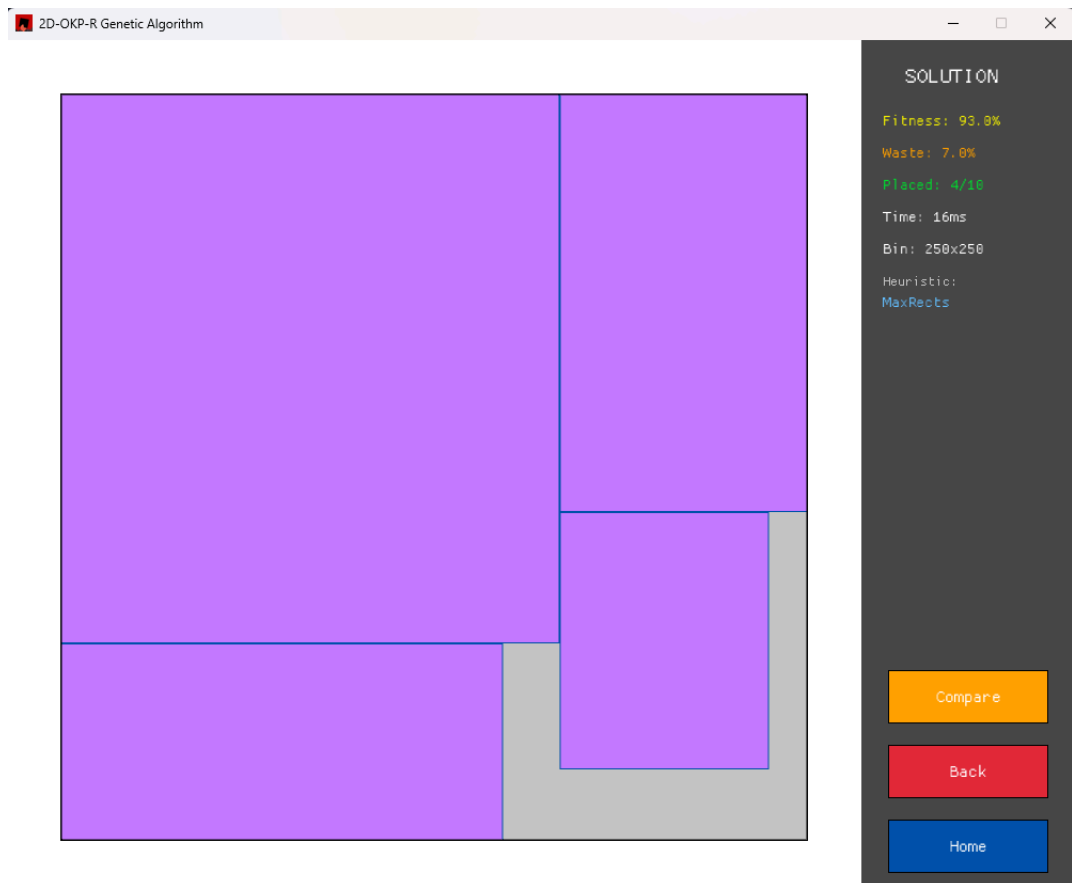
3.5 Екран за приказ резултата

Решење проблема се визуализује користећи вектор правоугаоника који се добија декодирањем хромозома изабраном хеуристиком. Свака хеуристика има своју функцију за декодирање и свој начин представљања решења. На екрану се приказује површина као велики правоугаоник, који је издељен на мање делове од којих сваки представља један распоређени правоугаоник.

Димензије површине се скалирају на основу димензија прозора како би се решење могло јасно и у целости видети. Површина се приказује великим сивим правоугаоником

или квадратом, док су постављени правоугаоници приказани као мањи правоугаоници љубичасте боје. Осим визуелизације решења, кориснику се приказује проценат искоришћености површине и отпада, број постављених правоугаоника, време извршавања (у милисекундама), димензије површине на коју се постављају правоугаоници и назив хеуристике која је коришћена за постављање правоугаоника.

Кликом на дугме “упоређи” (енг. *compare*) се прелази на екран за упоређивање хеуристика, где се добијено решење пореди са решењима добијеним коришћењем друге две хеуристике. На слици 10 приказан је екран за визуелизацију решења.



Слика 10: Визуелизација решења *gcut1* проблема.

3.6 Екран за поређење хеуристика

За одабран проблем приказана су два дијаграма, на првом је упоређена искоришћеност простора (енг. *fitness*) у процентима, а на другом време извршавања у милисекундама. На оба дијаграма су приказани резултати за све три хеуристике. Такође су приказане

визуализације решења за све три хеуристике које приказују како су постављени правоугаоници на површини.

Како се једна од хеуристика не би рачунала два пута и како би остало исто решење одабране хеуристике, приликом поређења покрећу се друге две хеуристике које претходно нису одабране за решавање проблема. На слици 11 приказан је екран за визуелизацију упоређивања хеуристика.



Слика 11: Визуелизација упоређивања хеуристика.

У оквиру рада решен је дводимензионални проблем ранца. Решење је реализовано коришћењем генетског алгоритма са три различите хеуристике за декодирање генома: *maximal rectangle*, *guillotine* и *skyline*. Тестирање је показало да су све три хеуристике биле ефикасне у решавању проблема, али су се разликовале у перформансама и квалитету решења. Унос података је могућ кроз JSON фајлове или ручно, што је омогућило флексибилност у коришћењу алата. Резултат је приказан кроз графички интерфејс, што је омогућило визуелизацију решења и лакше разумевање резултата. Решење је могуће упоредити са решењима добијеним другим хеуристикама, што је омогућило процену ефикасности и квалитета решења.

Решени проблеми обухватају одабир алгоритма за решавање проблема, моделовање хромозома, одабир хеуристика и декодирање генома помоћу њих, оптимизацију алгоритма, као и имплементацију и дизајн графичког интерфејса. Резултати на стандардним GCUT инстанцама показују да *maximal rectangle* у просеку даје највећу искоришћеност, *guillotine* је најбржи, док *skyline* представља компромис између квалитета и брзине.

Предности решења су флексибилан унос, јасна визуелизација и могућност поређења са другим хеуристикама.

Недостаци решења су што генетски алгоритам не гарантује оптимално решење и може бити осетљив на избор параметара. Такође, перформансе могу варирати у зависности од специфичних инстанци проблема.

Као правац даљег развоја може се размотрити адаптивно подешавање параметара генетског алгоритма, увођење додатних хеуристика за декодирање генома, као и проширење проблема тако да укључује тежине предмета или увођење више површина.

Списак слика

Слика 1	Архитектура генетског алгоритма.	8
Слика 2	Укрштање у две тачке.	11
Слика 3	Пример дељења површине.	13
Слика 4	Пример дељења површине гиљотинским сечењем.	15
Слика 5	Пример сегмената након постављања правоугаоника.	17
Слика 6	Почетни екран.	22
Слика 7	Екран за унос преко JSON фајла.	23
Слика 8	Екран за избор хеуристике.	24
Слика 9	Екран за ручни унос.	25
Слика 10	Визуелизација решења <i>gcut1</i> проблема.	26
Слика 11	Визуелизација упоређивања хеуристика.	27

Списак листинга

Листинг 1	Апстракција хеуристике преко енумерације.	6
Листинг 2	Пример JSON фајла.	7
Листинг 3	Иницијализација хромозома.	9
Листинг 4	Петља генетског алгоритма.	10
Листинг 5	Аутоматски избор хеуристике.	12

Списак табела

Табела 1	Поређење хеуристика на 13 <i>gsit</i> проблема.	18
----------	--	----

Списак коришћених скраћеница

Скраћеница	Опис
JSON	JavaScript Object Notation (формат за размену података)
2D	Дводимензионални
GCUT	Скуп проблема за тестирање алгоритама паковања и сечења
ms	Милисекунда

Списак коришћених појмова

Појам	Објашњење
Проблем ранца	Оптимизациони проблем у ком се бира подскуп предмета који се смешта у ограничен простор тако да се максимизује укупна вредност предмета. У овом раду вредност предмета се посматра кроз његову површину.
Декларација	Пријава дефиниције променљиве или функције у програмском коду.
<i>Crate</i>	Јединица организације кода у <i>Rust</i> програмском језику.
Модул	Логичка јединица организације кода унутар <i>crate</i> -а.
Енумерација	Тип који представља фиксни скуп вредности.
Структура	Комбинована колекција података која чини једну логичку целину.
Генетски алгоритам	Оптимизациони алгоритам инспирисан процесом природне селекције који се користи за решавање сложених проблема.
Хромозом	Бинарни низ који представља потенцијално решење у генетском алгоритму.
Ген	Основна јединица генетског кода која се налази у хромозому.
Популација	Скуп хромозома који представља могућа решења у генетском алгоритму.
<i>Fitness</i> функција	Функција која процењује квалитет решења у генетском алгоритму.
Укрштање	Операција у генетском алгоритму која комбинује два родитеља да би се створило потомство.
Мутација	Операција у генетском алгоритму која насумично мења део хромозома да би се одржала генетска разноликост.
Елитизам	Стратегија у генетском алгоритму која омогућава да се најбоља решења чувају из једне генерације у другу.
Хеуристика	Практично правило или стратегија која брзо налази добро, али не оптимално решење за сложене проблеме.
Декодирање хромозома	Процес претварања бинарног низа у корисно решење.
Benchmark	Стандардни скуп тест примера који се користи за процену перформанси алгоритама.

Кориснички ин-терфејс	Спољашњи део апликације који омогућава кориснику да комуницира са системом.
Скроловање	Операција којом се пролази кроз садржај који не улази у видљиви део прозора.

Биографија

Маша Иванов је рођена 04. октобра 2002. године у Новом Саду. Основну школу „Светозар Милетић“ завршила је у Тителу, а затим је уписала електротехничку школу „Михајло Пупин“ у Новом Саду, смер Информационе технологије. Године 2021. уписала је основне академске студије на Факултету техничких наука у Новом Саду, смер Софтверско инжењерство и информационе технологије.

Литература

- [1] S. M. M. M. Iori V.L. de Lima, “2DPackLib: a two-dimensional cutting and packing library.” Accessed: May 29, 2025. [Online]. Доступно на <https://site.unibo.it/operations-research/en/research/2dpacklib/>
- [2] A. Bhayani, “Solving the Knapsack Problem with Evolutionary Algorithms.” Accessed: May 29, 2025. [Online]. Доступно на <https://arpitbhayani.me/blogs/genetic-knapsack/>
- [3] G. D. Luca, “Roulette Selection in Genetic Algorithms.” Accessed: May 29, 2025. [Online]. Доступно на <https://www.baeldung.com/cs/genetic-algorithms-roulette-selection/>
- [4] J. E. Beasley, “Unconstrained guillotine cutting.” Accessed: May 29, 2025. [Online]. Доступно на <https://people.brunel.ac.uk/~mastjjb/jeb/orlib/gcutinfo.html/>